Report R1 — Tier 1a exact engine: construction and validation

Krylov sector analysis of quantum cellular automata

QCA Sector Analysis project (Tier 1)

2026-07-12 (engine_version `1a.1`)

Abstract

This report documents the exact Tier-1a pipeline that classifies quantum elementary cellular automata (QCA) by the fragmentation structure of their one-cycle evolution channel, and the validation gate it has passed. The engine computes the directed one-cycle transition graph on the 2^N computational basis states using *exact* $\mathbb{Z}[1/\sqrt{2}]$ amplitude arithmetic (no floating point, no tolerances), extracts strongly connected components (Krylov sectors for unitary rules), and records sector sizes and basin structure. Three independent checks all pass: (i) the one-step support $\mathrm{succ}(x)$ agrees with a dense brick-wall unitary oracle on 3840 checks; (ii) the eight ground-truth regression items of the specification are reproduced exactly, including rule 150 open-boundary sector sizes up to $N = 17$; (iii) 35/35 sector-size cross-checks against the independent Julia `HSF` code match exactly. We also show that, for the one-cycle transition graph, the exact arithmetic is provably *redundant* — a single brick-wall cycle cannot produce interference, so naive support propagation gives the identical graph on all 258,048 tested images — yet is kept because it certifies the branch decomposition and is load-bearing downstream (Tier 1b/1d). Finally we report a physically instructive finding on the spin-flip (inversion) symmetry of rules 201 and 108.

1 Model and conventions

N qubits sit on a line, sites $i = 0, \ldots, N-1$, site 0 the least significant bit. A classical ECA rule $R \in \{0, \ldots, 255\}$ is recoded into a local-channel tuple $(r_{00}, r_{01}, r_{10}, r_{11})$ with each $r_{mn} \in \{\mathtt{I}, \mathtt{V}, \mathtt{D}, \mathtt{E}\}$ indexed by the neighbour pair $(m, n) = (x_{i-1}, x_{i+1})$:

$$(c_p, c_q) = (0,1) \to \mathtt{I}, \quad (1,0) \to \mathtt{V}, \quad (0,0) \to \mathtt{D}, \quad (1,1) \to \mathtt{E},$$

where $c_p = c_{4m+n}$, $c_q = c_{4m+n+2}$ are the rule's outputs for centre 0/1. The unitary gate is $\mathtt{V} = H$ (Hadamard); $\mathtt{D}, \mathtt{E}$ are the reset (decay / excitation) channels used only for dissipative rules (Tier 1b onward). One QCA cycle is the brick-wall product: apply the local update at all *even* sites, then at all *odd* sites, controls reading the current (mid-cycle) bitmask. Two boundary conventions are implemented and always recorded: periodic (`pbc`) and open with fixed vacuum padding $x_{-1} = x_N = 0$ (`obc0`).

A rule is *unitary* iff every $r_{mn} \in \{\mathtt{I}, \mathtt{V}\}$; there are exactly $2^4 = 16$ such rules, the subject of this report.

1.1 Exact amplitude arithmetic

Every amplitude produced by conditional Hadamards lies in $\mathbb{Z}[1/\sqrt{2}]$. We store a branch as a dictionary $\{x : (a, b)\}$ with one shared dyadic exponent m, amplitude $(a + b\sqrt{2})/2^m$, a, b arbitrary-precision integers. A conditional-H layer sets $m \to m + 1$ and updates non-fired entries $(a, b) \to (2a, 2b)$, fired entries $(a, b) \to \pm(2b, a)$; addition is componentwise; the exact-zero test is $a = b = 0$ (correct because $\sqrt{2}$ is irrational). No tolerance is used anywhere in the graph stage. Unitary branch norms evaluate to exactly 1 (as `Fractions`).

2 Directed graph and sectors

Nodes are bitmasks; edges are implicit via the memo-free $\mathrm{succ}(x) = \{y : \langle y|\Phi_C(|x\rangle\langle x|)|y\rangle \neq 0\}$, the union of Kraus-branch supports after one cycle. Sectors are the strongly connected components (SCCs) of this *directed* graph. For unitary rules the channel is doubly stochastic, so every state is recurrent and each weakly connected component is a single SCC; we therefore extract sectors with a memory-light union–find over streamed edges (one `succ` pass, $O(2^N)$ integers). This reduced the peak memory of rule 150 at $N = 17$ from 2073 MB (stack-based Tarjan) to 45 MB at equal runtime, and matches the directed Tarjan result on all 16 unitary rules. An ergodic early-exit flags a rule ”effectively ergodic at this N” when one sector exceeds a fraction f_{erg} of the space.

3 Is the exact arithmetic necessary? A one-cycle no-interference theorem

The engine tracks exact $\mathbb{Z}[1/\sqrt{2}]$ amplitudes and prunes any that cancel to *exactly* zero. A cheaper engine would just propagate the reachable *support*: treat each `V` as branching a site to both bit values, $\{x\} \to \{x \wedge \overline{2^i},\, x \vee 2^i\}$, take unions, and never look at a sign. Since the naive support always contains the exact support, the only thing the amplitudes can buy is the *deletion* of an edge $x \to y$ whose amplitude $\langle y|\Phi_C|x\rangle$ vanishes by destructive interference. For the 16 unitary rules — where the sector partition is the entire deliverable — does this ever happen?

Empirically, never. Comparing $\mathrm{succ}(x)$ against the naive support propagation over every basis state, both boundary conventions and $N = 3, \ldots, 8$: for the 16 unitary rules the two agree on all 16,128 one-cycle images and the union–find sector partitions are identical; extending the sweep to all 256 rules (per Kraus branch) gives 0 mismatches on 258,048 images. No edge is ever interference-cancelled within one cycle.

Why — a depth-1 argument. A cycle is two half-layers, the even sublattice then the odd. Within a half-layer every gate acts on a distinct qubit of the sublattice being updated, and each gate's control reads only the *complementary* sublattice, which that half-layer does not touch. Conditioned on the (definite) control background, the half-layer is thus a tensor product of single-qubit gates on disjoint qubits — a depth-1 circuit — and a product of single-qubit gates never makes two computational paths reconverge: each output configuration has a unique preimage. Concretely, cancellation on a basis state y requires two source states differing in exactly the bit a `V` just acted on to be simultaneously present with equal or opposite amplitude; but before site i's gate fires that bit is still definite (the input value in the even layer, the post-even value in the odd layer), so the partner is absent, and once site i branches no later gate in the layer touches bit i. Hence $\mathrm{succ}(x)$ equals the naive support *exactly*, for every rule and every N: for building the one-cycle transition graph, the $\mathbb{Z}[1/\sqrt{2}]$ ring is provably redundant.

The guard is not vacuous. It is the once-per-site brick-wall structure, not the arithmetic, that removes interference. Touch a qubit twice in one step and it returns: $H{\cdot}H = \mathbb{1}$, so applying `V` to a fresh $|0\rangle$ and then `V` again yields exact support $\{0\}$ — the $|1\rangle$ amplitude $(0,1)+(0,-1)$ cancels to $(0,0)$ and is pruned — whereas the naive rule would report $\{0,1\}$. A deeper local circuit, a two-site gate, or a non-brick update order would all reintroduce cancellation.

Why we keep it anyway. Three reasons, none of them the transition graph. (i) *Certification.* The exact amplitudes let the branch decomposition check itself: unitary branch norms evaluate to exactly 1 and dissipative Kraus probabilities sum to exactly 1 (as `Fractions`), so a bug in the branching is caught immediately — a guarantee floating point cannot give. (ii) *Downstream exactness.* Tier 1b/1d build the restricted superoperator and the pair graph from these same branches, and there the amplitude *values* (not merely their support) are load-bearing: the Césaro rank and the peripheral spectrum are exact functions of the $\langle y|K_b|x\rangle$. (iii) *Architecture-agnostic correctness.* The no-interference property is a theorem about depth-1 brick-wall layers, established *post hoc*; the engine does not assume it. Keeping the ring makes the pipeline correct by construction rather than correct by a lucky architecture, and it would flag interference the moment the model changed.

4 Validation

4.1 Dense brick-wall oracle

For every unitary rule, $N = 3, \ldots, 6$, both boundary conventions, and every basis state, the support of the corresponding column of the exactly-built dense unitary equals $\mathrm{succ}(x)$: **3840/3840 checks pass** (oracle unitarity error $< 10^{-15}$).

4.2 Ground-truth regression table (specification §8)

All eight items pass (`pytest`, 68 tests green):

Rule	Claim	Result
204 (`IIII`)	2^N singleton SCCs	pass, both bc
51 (`VVVV`)	single SCC of size 2^N	pass, both bc
150, `obc0`	domain-wall sectors $\binom{N+1}{w}$, w even	pass, $N = 8$–17
54	one frozen $\|0\rangle$ + one giant SCC of size $2^N - 1$	pass, $N \le 12$
156, `pbc`	# sectors $= L_N + 1$ (Lucas)	pass, $N = 3$–11
22, `pbc` even N	3 recurrent classes, each size 1	pass
0,22,28,29	dissipative branch norms = 1 exactly	pass

For rule 150 at $N = 17$, `obc0`, the exact sector sizes are

$$[43758, 43758, 18564, 18564, 3060, 3060, 153, 153, 1, 1] = \{\, \tbinom{18}{w} : w \text{ even} \,\},$$

reproduced bit-for-bit.

4.3 Cross-check against the `HSF` Julia oracle

The pre-existing `HSF` code builds the sparse unitary and extracts Krylov sectors by connected components on the symmetrised adjacency (valid for unitary rules). Its `_H_ SectorData` files use `V` $= H$, `obc0`. Matching its rule numbering (HSF 0–15; 6=150, 1=201, 2=198, 4=156, 8=108), **35/35 (rule, N) sector-size multisets match exactly** for $N = 8$–14, and rule 150 additionally at $N = 15, 16, 17$. Representative largest-sector ($D_{\max}$) sequences (`obc0`):

Wolfram (HSF)	$D_{\max}(N)$	character
150 (6)	$126, 210, 462, 924, 1716, \ldots, 43758$	central binomials
201 (1)	$55, 89, 144, 233, 377, 610, \ldots$	Fibonacci (φ)
108 (8)	$21, 34, 55, 89, 144, \ldots$	shifted Fibonacci
156 (4) = 198 (2)	$12, 16, 20, 27, 36, 48, 64, \ldots$	base $\approx 4^{1/5}$

That 156 and 198 share sector data confirms the reflection reduction (they are a reflection pair). The Fibonacci $D_{\max}$ of 201 is the Floquet-PXP golden-ratio scaling.

5 Symmetry note: rules 201 vs. 108 (broken inversion)

Rules 201 = ($\texttt{V}, \texttt{I}, \texttt{I}, \texttt{I}$) and 108 = ($\texttt{I}, \texttt{I}, \texttt{I}, \texttt{V}$) are $0 \leftrightarrow 1$ spin-flip (complement) partners. Classically the complement is a symmetry, but here the flip also conjugates the gate by X, and $XHX \neq \pm H$. We find, exactly:

- `pbc`: 201 and 108 have *identical* sector sizes for all N tested $(47, 76, 123, 199, 322, 521, \dots)$. Because $|\langle y|XHX|x\rangle| = |\langle y|H|x\rangle|$, the bit-flip $X^{\otimes N}$ (a basis permutation) preserves the support graph, hence the sector partition.
- `obc0`: 201 and 108 *differ* ($D_{\max}$ Fibonacci vs. shifted Fibonacci, as in `HSF` rules 1 vs. 8). The fixed vacuum-0 padding is mapped to vacuum-1 by the bit-flip, so the boundary breaks the symmetry.

The full dynamics (spectra, entanglement) is spin-flip-broken *everywhere* because $XHX \neq \pm H$ changes amplitude signs. Consequently the pipeline never reduces `V`-containing rules by spin-flip. Reflection sector-size invariance is likewise used only for unitary rules; under the even-first brick-wall convention it does not extend to dissipative rules (reflection swaps the two layers).

6 Reproducibility

The engine is deterministic (no RNG in Tier 1a). Results are append-only JSON lines `results/{rule}_{bc}.js` keyed by (`engine_version`, rule, N, bc), with an audit trail in `checkpoints/manifest.jsonl`; a killed run resumes without recomputation. Regression items are encoded as `pytest` tests under `code/qca_fragmentation/tests/`. This report rests on engine_version `1a.1`.